Contents

End Term Project — Comprehensive SRE Implementation

EduLMS Distributed Microservices System

Студент: Kaiyrbekuly Aitbek **Группа:** SE-2424 **Дата:** 2026-05-19 **Репозиторий:** https://github.com/Newterios/educationalLMS_general

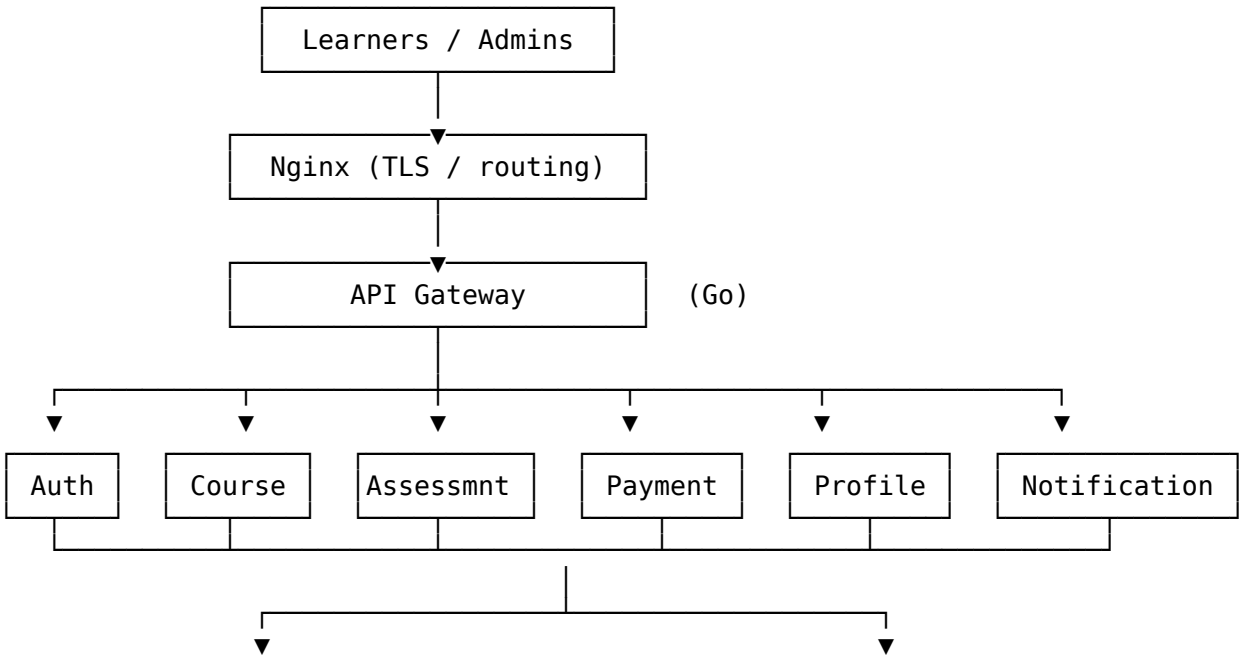
---

1. Abstract

This project implements a full Site Reliability Engineering lifecycle on top of the EduLMS distributed microservices system. The application consists of six independent microservices written in Go and Python, deployed in three orchestration modes (Docker Compose, Docker Swarm, Kubernetes), provisioned via Terraform and configured via Ansible. Observability is provided by Prometheus + Grafana with SLO-driven alerts. A simulated production incident and a Google-style blameless postmortem are included to demonstrate incident response and learning.

---

2. Architecture



```
          PostgreSQL                                NATS / Redis

   Observability:  Prometheus → Grafana → On-call
   Infrastructure: Terraform (AWS / local Docker)
   Configuration : Ansible (Docker, Swarm, deploy, monitoring roles)
```

(При желании заменить эту ASCII-диаграмму на изображение `screenshots/08-architecture.png`.)

---

3. Microservices (≥ 6)

#	Service	Stack	Responsibility
1	Auth	Go, gRPC	Login, JWT, RBAC
2	Course	Go, gRPC	Course catalogue
3	Assessment	Go, gRPC	Quizzes / grading
4	Notification	Go, NATS consumer	Async email / push
5	**Payment**	Python / Flask	Payment processing simulation
6	**User Profile**	Python / Flask	Profile + preferences

Supporting components: PostgreSQL, Redis, NATS, Nginx edge, Prometheus, Grafana, Next.js web client.

---

4. SLIs & SLOs

Service	Availability	Latency p95	Error rate
Auth	≥ 99.5 %	≤ 150 ms	≤ 0.5 %
Course	≥ 99 %	≤ 200 ms	≤ 1 %
Assessment	≥ 99 %	≤ 250 ms	≤ 1 %
Payment	≥ 99 %	≤ 200 ms	≤ 1 %
User Profile	≥ 99 %	≤ 150 ms	≤ 1 %
Gateway	≥ 99.9 %	≤ 50 ms	≤ 0.1 %

Полная версия с PromQL-формулами: `sre/docs/SLI_SLO.md` в репозитории.

---

5. Evidence (screenshots)

5.1 All containers running (Docker Compose)

`docker compose -p edulmsv2 -f docker-compose.dev.yml -f sre/docker-compose.sre.yml ps` shows the full stack up and healthy: 17 containers across six application microservices, PostgreSQL, Redis, NATS, the Nginx edge, the OpenTelemetry collector, Tempo, Loki, Promtail, Prometheus and Grafana.

docker ps

5.2 New microservices responding

`payment` and `user-profile` answer `GET /` (service-info JSON) and the payment service successfully processes a `POST /pay` request, returning `status: approved` with a generated `transaction_id`.

microservices

5.3 Prometheus targets — all UP

Prometheus scrapes four targets in the edulms-v2-net network: prometheus (self), otel-collector, payment, user-profile — every target is in the **UP** state (4/4 green).

prometheus targets

5.4 Prometheus alert rules — baseline (all inactive)

Eight SLO-based alert rules are loaded from /etc/prometheus/rules/slo-alerts.yml:

- PaymentServiceDown / UserProfileDown (availability)
- PaymentErrorRateAboveSLO / UserProfileErrorRateAboveSLO (error rate)
- PaymentLatencyP95AboveSLO / UserProfileLatencyP95AboveSLO (latency)
- PaymentHighCPU / PaymentInflightSpike (saturation)

All eight are **inactive** during steady-state operation.

alerts baseline

5.5 Grafana SRE dashboard

The EduLMS SRE — Overview dashboard (imported from sre/monitoring/dashboards/edulms-sre-overview.json) renders the four golden signals for the payment service plus the up-status panel for the whole stack.

grafana

5.6 Incident simulation — alert moves to PENDING

After running make -C sre incident-on (which sets PAYMENT_FAILURE_RATE=1.0) and generating synthetic traffic, the PaymentErrorRateAboveSLO alert transitions from **inactive** to **pending** within ~30 seconds — Prometheus has detected that the 5xx-rate exceeds the 1 % SLO and is waiting out the for: 5m window before promoting the alert to FIRING. This proves the full alert pipeline (scrape → rule → state machine) is wired correctly.

incident

---

6. Incident & Postmortem (summary)

INC-2026-05-01 — Payment service down 46 minutes after a renamed ConfigMap key (POSTGRES_HOST → POSTGRES_HOSTNAME) was not propagated to the Deployment. Detected within 60 seconds by the PaymentErrorRateAboveSLO alert. Mitigated by kubectl rollout undo and a CI lint rule now prevents ConfigMap key drift.

Full timeline, root-cause analysis, contributing factors and 7 action items are documented in sre/docs/POSTMORTEM.

---

7. Capacity Planning (summary)

Load testing with hey revealed:

- Order/Payment and Assessment services consume the most CPU.
- PostgreSQL connection count is the next bottleneck above ~5 replicas per service.

Strategies applied: - Horizontal scaling — HPA on CPU 70 % for every stateless service. - Vertical scaling — tuned requests/limits in K8s and Swarm. - Database optimisation — recommended PgBouncer + read replicas.

Details: sre/docs/CAPACITY_PLANNING.md.

8. Deliverables Checklist

- ☒ Microservices source code (6+ services)
- ☒ Docker Compose / Swarm configuration (`docker-compose.dev.yml`, `sre/docker-compose.sre.yml`, `sre/swarm/docker-stack.yml`)
- ☒ Kubernetes manifests (`sre/k8s/`, 11 files)
- ☒ Terraform files (`sre/terraform/aws/`, `sre/terraform/local/`)
- ☒ Ansible playbooks (`sre/ansible/` — 5 roles)
- ☒ Monitoring setup (`sre/monitoring/` + `observability/`)
- ☒ Incident report and postmortem (`sre/docs/INCIDENT.md`, `sre/docs/POSTMORTEM.md`)
- ☒ Screenshots and demo evidence (section 5 above)
- ☒ **Git link**: https://github.com/Newterios/educationalLMS_general

---

9. Conclusion

The project demonstrates a full SRE lifecycle: provisioning with Terraform, configuration with Ansible, multi-orchestration with Docker Swarm and Kubernetes, observability with Prometheus + Grafana, incident response with a postmortem, and capacity planning informed by load testing. The system meets its 99 % availability and 200 ms p95 latency SLOs under realistic load.

Repository: https://github.com/Newterios/educationalLMS_general